Step-Indexed Hoare Logic as Ramified Forcing

Anonymous Author(s)

Abstract

We give two technical observations about step-indexed Hoare logic, made mechanically precise by re-presenting the technique as *ramified forcing*: Cohen's original forcing construction stratified by a well-founded hierarchy of levels.

First, we exhibit a structural diagnostic that distinguishes when the internal Löb rule is the natural soundness tool for a recursive Hoare-style rule. For the `while` rule of IMP, internal Löb and meta-level induction on the step index produce structurally identical proofs. We mechanize both side by side. For the `fix` rule of an untyped λ-calculus, the soundness predicate is itself recursive in the program being verified. This is a difference in what we call the *recursion locus*, and only the internal Löb rule plays to that structure. This is, to our knowledge, the first explicit and mechanized articulation of when the internal-Löb idiom of Iris and related systems is essential versus when it is a presentational choice.

Second, we characterize exactly when Löb's rule is available: in any forcing-style pre-structure satisfying the order-theoretic axioms of our abstract framework, Löb's rule holds for every iProp if and only if the strict refinement relation is well-founded. The forward direction is the standard well-founded-induction proof. The backward direction uses the forcing-accessibility predicate $\varphi_{\mathrm{acc}}\, p := \mathtt{Acc}\ \mathtt{lt}\ p$ as a single internal iProp witness, so that a universal Löb assumption forces well-foundedness of the strict relation. A concrete ill-founded instance over $\mathbb{Z}$ realises one direction constructively. This characterization is a step-indexed-Hoare-logic counterpart of a classical fact about Gödel–Löb modal logic. The iff form, stated and mechanized in the step-indexed Hoare logic setting, does not, as far as we can determine, appear in the verification literature.

The two observations rest on a forcing-style representation of step-indexed Hoare logic that we develop in Rocq alongside them: a propositional layer, Hoare logic over IMP and over a λ-calculus with `fix`, an abstract framework parametric in the forcing notion, and a propositional-fragment embedding of meta-level Prop into the Jaber–Tabareau–Sozeau forcing translation of CIC. The complete development is mechanized in Rocq with no external dependencies.

CCS Concepts: • **Theory of computation → Modal and temporal logics**; **Program semantics**; **Hoare logic**; *Type theory*.

Keywords: step-indexed semantics, forcing, ramified forcing, Hoare logic, Löb's rule, Rocq mechanization, Gödel–Löb logic, forcing translations

1 Introduction

Step-indexed reasoning has become the standard semantic technique for verifying higher-order programming languages with mutable state, recursive types, and concurrency. From Appel and McAllester's original step-indexed model of recursive types [Appel and McAllester 2001] to the modern Iris ecosystem [Jung et al. 2018], the same basic idea recurs: instead of asking whether a program satisfies a specification, ask whether it satisfies the specification at every finite observation depth n, then recover the unindexed statement as the limit. Each level of the hierarchy is defined without circularity by appeal only to lower levels, and a "later" modality $\triangleright$ mediates the descent from one level to the next.

What the step-indexed literature does not usually say is that this technique is, in classical set-theoretic terms, *forcing*. Cohen introduced forcing in 1963 to prove the independence of the continuum hypothesis [Cohen 1963, 1966]. In its original presentation, called *ramified forcing* after the ramified type theory of Russell and Whitehead [Russell and Whitehead 1913], the construction of the generic extension is stratified by a well-founded hierarchy of levels, with recursion at level α permitted to consult only levels strictly below α. Shoenfield's 1971 reformulation [Shoenfield 1971] eliminated the ramification for the set-theoretic application: the modern textbook presentation of forcing does not use it. But that structure is exactly the structure of step-indexed logic: well-founded stratification, level descent, and a forcing relation $p \Vdash \varphi$ monotone in the strength of p. The forcing conditions are the natural numbers ω with $\leq$, the relation $n \Vdash \varphi$ is monotone in n, the modality $\triangleright$ is the level descent operator, and Löb's rule is well-founded induction on the forcing order, internalized into the logic.

Contributions. We give two technical observations about step-indexed Hoare logic, together with the forcing-style framework that makes them stateable. All claims are mechanized in Rocq.

1. **A structural diagnostic for the internal Löb rule** (Section 5). Recursive Hoare-style rules come in two kinds, distinguished by the *recursion locus* of the soundness predicate. For the IMP `while` rule (Section 3), the soundness predicate is recursive only in the step index. Outer induction on n and an appeal to the internal Löb rule produce structurally identical proofs, and we mechanize both (`hoare_while`, `hoare_while_iLob`). For the `fix` rule of an untyped λ-calculus (Section 4), the soundness predicate is recursive in the program itself. Meta-level induction on n remains available but no longer plays to the

predicate's structure, while internal Löb does. This articulation does not, to our knowledge, appear in either the synthetic guarded domain theory literature or in the Iris literature, where internal Löb is heavily used but the question of when it is essential is not explicitly diagnosed.

2. **A characterization of Löb's rule by well-foundedness** (Section 7). In any forcing-style pre-structure satisfying the order-theoretic axioms of our abstract framework, Löb's rule holds for every iProp iff the strict refinement relation is well-founded. The forward direction is the standard well-founded-induction proof, replayed in the pre-structure. The backward direction uses the *forcing-accessibility* predicate $\varphi_{\mathrm{acc}}\, p \;:=\; \texttt{Acc}\,\texttt{lt}\, p$ as an internal iProp witness whose Löb premise is forced everywhere, so a universal Löb assumption forces every condition to be accessible. A concrete would-be forcing structure over $\mathbb{Z}$ where every iProp construction goes through but Löb's rule fails on $\bot$ realises one direction constructively. This is a step-indexed Hoare logic counterpart of a classical fact about Gödel–Löb modal logic [Boolos 1993; Smoryński 1985] that, in the iff form stated and mechanized in the verification setting, does not appear in the literature we surveyed. Methodologically, the well-foundedness of the forcing notion is constitutive of the modal logic, not a presentation choice that can be lifted away by a clever recursion on rank.
3. **A mechanized forcing-style re-presentation of step-indexed Hoare logic** (Sections 2–4), together with an **abstract forcing framework parametric in the forcing notion** (Section 6). The abstract framework instantiates at $(\omega, \leq)$ to recover the standard step-indexed semantics and at $(\omega \times \omega, \leq_{\mathrm{pw}})$ to demonstrate genericity. Ordinals up to a bound, including the index structure of Transfinite Iris [Spies et al. 2021], are further instances. Löb's rule, in the abstract setting, is proved by `well_founded_ind` on the strict refinement relation, making precise the equivalence between the internal Löb rule and well-founded induction on the forcing order.
4. **An embedding of meta-level** Prop **into the propositional fragment of the Jaber–Tabareau–Sozeau forcing translation of CIC** [Jaber et al. 2012] (Appendix A). The embedding is a Heyting algebra homomorphism. The later modality is the step-indexed-specific content lying outside its image. A full mechanization of the type-level translation is future work.

What is and is not novel. The re-presentation itself is not novel. The topos-of-trees account of step-indexing [Birkedal et al. 2012] is implicitly a forcing-style semantics (Section 8.1). The Iris ecosystem uses internal Löb without referring to it as forcing. The JTS translation produces, at the propositional level, structures that coincide with our `iProp`. The framework we give is the smallest such account that supports the two technical observations above. Its purpose is to expose the structure inside these existing constructions that makes the diagnostic and the counterexample stateable in the first place.

Scope. We work at the level of plain Hoare triples and a partial-correctness wp predicate, with no separation logic, heap, or concurrency. The λ-calculus of Section 4 is untyped, and our recursive spec takes a single predicate S as both pre- and post-condition (the rule generalizes to distinct pre/post in the formalization).

Outline. Section 2 gives the propositional layer. Section 3 develops Hoare logic for IMP and proves the `while` rule by meta-level induction. Section 4 develops Hoare logic for an untyped λ-calculus with `fix` and proves the soundness rule via the internal Löb rule. Section 5 articulates the diagnostic. Section 6 gives the abstract framework. Section 7 characterizes Löb's rule in terms of well-foundedness of the forcing notion. Section 8 positions the contribution against the literature, and Appendix A formalizes the JTS embedding. The complete development is mechanized in Rocq with no external dependencies.

2 The Propositional Layer

Before defining a Hoare logic we need a propositional logic to express the pre- and post-conditions in. To make explicit the connection to forcing, we build this propositional logic as a Kripke-style forcing semantics: a proposition is not a single truth value but a *family* of truth values indexed over a poset of *conditions*, and entailment is forcing at every condition.

The simplest forcing notion that supports step-indexed reasoning is the natural numbers with the usual order. Each natural number stands for an "observation depth": condition n records that we have consumed n program steps and committed to a partial view of the computation. Following the topos-of-trees and Iris convention, we take *smaller* naturals to be *stronger* conditions: 0 is the weakest condition (we have observed nothing and are committed to nothing), and larger n are stronger (we are committed to a deeper approximation). Under this orientation, the forcing extension order is the converse of $\leq_{\mathbb{N}}$, and a forcing-style proposition must be *downward closed* in $\leq_{\mathbb{N}}$: a proposition forced at depth n is forced at every shallower depth $m \leq n$.

```
Definition cond : Type := nat.

Record iProp : Type := MkProp {
  ihold : cond → Prop;
  imono : forall n m, m ≤ n → ihold n →
      ihold m
}.
```

```
Notation "n ⊩ φ" := (ihold φ n).
```

The notation $n \Vdash \varphi$ matches the classical forcing notation, and it is more than analogical: an `iProp`, taken with its `imono` field, is exactly a presheaf on $(\omega, \leq)$, hence an object of the topos of trees [Birkedal et al. 2012], which is exactly the Kripke model of intuitionistic logic with worlds in ω, and hence (Section 8.1) the forcing-style semantics with conditions in ω.

Entailment is forcing at every condition:

```
Definition ientails (φ ψ : iProp) : Prop
    :=
  forall n, n ⊩ φ → n ⊩ ψ.

Notation "φ ⊢ ψ" := (ientails φ ψ).
```

Connectives. The Heyting-algebra connectives have the standard Kripke interpretation. Truth and falsity are the constant predicates. Conjunction and disjunction are pointwise. Implication is the Kripke implication, quantified over all stronger conditions:

```
Definition iAnd (φ ψ : iProp) : iProp :=
  MkProp (fun n => n ⊩ φ /\ n ⊩ ψ) (
      iAnd_mono φ ψ).

Definition iImpl (φ ψ : iProp) : iProp :=
  MkProp (fun n => forall m, m ≤ n → m ⊩
      φ → m ⊩ ψ)
        (iImpl_mono φ ψ).
```

The quantification "$\forall m \leq n$" in `iImpl` is the characteristic move of intuitionistic Kripke semantics. Without it, the framework would collapse to classical logic with a step index. The monotonicity proofs `iAnd_mono` and `iImpl_mono` are routine. In each case the proof simply transports the membership witness through the order.

The later modality. The genuinely new ingredient over the standard Kripke interpretation of intuitionistic logic is the *later* modality $\triangleright$. It is the operator that says "to talk about a proposition here, you must drop one level". In our $(\omega, \leq)$ orientation, $\triangleright\varphi$ at depth n holds if $n = 0$ (we are at the weakest condition and the modality is vacuously satisfied) or if φ holds at the shallower depth $n - 1$:

```
Definition iLater (φ : iProp) : iProp :=
  MkProp (fun n => match n with
                  | 0 => True
                  | S k => k ⊩ φ
                  end) (
                      iLater_mono_helper φ
                      ).

Notation "▹ φ" := (iLater φ).
```

Two readings of this operator are worth stating explicitly. In step-indexed-logic terms, $\triangleright\varphi$ is "φ when we look at the computation one step later", and the shift is what guards self-referential definitions against circularity. In forcing terms, $\triangleright$ is the explicit level-descent operator: at level $n + 1$ we may consult level n. This is precisely the ramification of Cohen's original construction (Section 8, "Forcing in set theory"): the recursive definition of forcing at level α may invoke forcing only at strictly lower levels, and $\triangleright$ is the syntactic device that makes the descent explicit.

Heyting algebra rules. With the connectives in hand, the standard intuitionistic introduction and elimination rules become direct consequences of their Kripke definitions. We mechanize the full set:

```
Lemma iAnd_intro φ ψ χ : φ ⊢ ψ → φ ⊢ χ →
      φ ⊢ ψ ∧ χ.
Lemma iAnd_proj_l φ ψ : φ ∧ ψ ⊢ φ.
Lemma iOr_intro_l φ ψ : φ ⊢ φ ∨ ψ.
Lemma iOr_elim φ ψ χ : φ ⊢ χ → ψ ⊢ χ → φ
      ∨ ψ ⊢ χ.
Lemma iImpl_intro φ ψ χ : φ ∧ ψ ⊢ χ → φ ⊢
      (ψ → χ).
Lemma iImpl_elim φ ψ : (φ → ψ) ∧ φ ⊢ ψ.
```

Each proof simply moves the assumed monotonicity through the relevant connective. The work is small but worth doing mechanically because it confirms that the framework is genuinely a Heyting algebra without any informal appeal to topos-theoretic abstractions.

The basic laws of the later modality follow the same pattern. We prove that $\triangleright$ is monotone in the entailment order, that any proposition entails its own later ($\varphi \vdash \triangleright\varphi$), that $\triangleright$ distributes over $\wedge$ in both directions, and that $\triangleright\top$ is interderivable with $\top$.

Löb's rule. The technical heart of the propositional layer is Löb's rule:

Theorem 1 (Löb's rule). *For every* `iProp` φ, $(\triangleright\varphi \to \varphi) \vdash \varphi$.

We give the rule the standard step-indexed proof, by induction on n:

```
Lemma iLob φ : (▹ φ → φ) ⊢ φ.
Proof.
  intros n Hn.
  induction n as [|n IH].
  - apply (Hn 0); [lia | exact I].
  - apply (Hn (S n)); [lia | simpl].
    apply IH.
    intros m Hm Hlate.
    apply (Hn m); [lia | exact Hlate].
Qed.
```

It is worth noticing what this proof actually uses. The base case relies on the fact that $\triangleright\varphi$ at $n = 0$ is *vacuously*

True, while the inductive step uses the hypothesis at $n+1$ together with the induction hypothesis at n. In other words, Löb's rule is, at the meta level, well-founded induction on the forcing order ω. This observation will return in Section 6: we will see there that in the abstract framework Löb's rule *is* `well_founded_ind` on the strict refinement relation.

Step-indexing is not collapsing. A useful sanity check is that the later modality changes the propositional theory: it is not the case that $\triangleright\bot$ entails $\bot$. At $n = 0$, the proposition $\triangleright\bot$ is `True` by definition, whereas $\bot$ is `False`, so the entailment fails at $n = 0$:

```
Lemma iLater_False_distinct : ~ (▷ ⊥ ⊢ ⊥).
Proof. intros H. apply (H 0); simpl; exact
    I. Qed.
```

This is the canonical witness that step-indexing genuinely extends the propositional theory, rather than collapsing to ordinary intuitionistic logic. We will revisit it in Section 7 to make the converse point: if we *drop* the well-foundedness of the forcing order, then $\triangleright\bot$ becomes equivalent to $\bot$ in a way that breaks Löb's rule entirely.

3 Hoare Logic for IMP

We build a Hoare logic for a small while-language IMP, calibrated for the conceptual point of this paper: showing how the forcing-style propositional layer lifts to a sound Hoare logic, and proving the `while` rule both by meta-level induction on the step index and by an explicit appeal to the internal Löb rule. Exhibiting both proofs side by side makes precise the claim that "Löb's rule *is* induction on the step index." IMP has no higher-order features, no allocation, and no concurrency, so the step indices are not yet doing technical work.

IMP commands and their small-step semantics are standard:

```
Inductive cmd : Type :=
  | CSkip | CAssign : var → aexp → cmd
  | CSeq : cmd → cmd → cmd
  | CIf : bexp → cmd → cmd → cmd
  | CWhile : bexp → cmd → cmd.
```

Each small step corresponds to one decrement of the step-index clock that the wp predicate tracks.

Step-indexed wp and the Hoare triple. The wp predicate is the step-indexed analogue of the Dijkstra wp: `wp_at` $n\, c\, Q\, s$ says that running c from s for at most n steps is safe and any final state satisfies Q. In IMP, every non-`CSkip` command can step, so the safety condition is automatic and we do not record it explicitly. The invisibility of safety in IMP is what makes this section a calibration, and Section 4 is where it becomes visible.

```
Fixpoint wp_at (n : nat) (c : cmd)
               (Q : state → Prop) (s :
                   state) : Prop :=
  match n with
  | O => True
  | S k => (c = CSkip → Q s) /\
           (forall c' s', step (c, s) (c',
                s') → wp_at k c' Q s')
  end.
```

The predicate is downward-closed in n and contravariantly monotone in Q, and lifts to an `iProp` via `MkProp`. A Hoare triple is the `iProp` saying "for every s with $P\, s$, the wp of c targeting Q holds." Validity, written `hoare_valid` $P\, c\, Q$, is the entailment $\top \;\vdash\;$ `hoare` $P\, c\, Q$. Equivalently, by `hoare_valid_alt`, it is the meta-level statement "for every n and s with $P\, s$, the n-step-bounded wp holds."

Partial-correctness Hoare logic has five structural rules: skip, assignment, sequencing, conditional, and consequence. Each becomes a small lemma about `wp_at`, provable by routine unfolding at $n+1$ and inversion on the available step. Sequencing requires a small auxiliary lemma `wp_seq` decomposing the wp of `CSeq c1 c2` into the wp of `c1` targeting the wp of `c2`, with one use of `wp_at_mono` to align indices. None of these five rules uses the propositional layer. We could state them entirely at the meta level. The propositional layer pays off for `while`.

The `while` rule, meta-level induction. Soundness of $\{P \wedge b\}\, c\, \{P\} \;\vdash\; \{P\}$ `while` $b\, c\, \{P \wedge \neg b\}$ says that a sound loop body gives a sound loop, and the proof goes by induction on the step index. The base case is trivial. The inductive step unfolds the wp at $n+1$, case-analyses on the step, and uses the induction hypothesis to discharge the recursive `while` obligation:

```
Lemma hoare_while P b c :
  hoare_valid (fun s => P s /\ beval s b =
      true) c P →
  hoare_valid P (CWhile b c)
                (fun s => P s /\ beval s b =
                    false).
Proof.
  intros HC. rewrite hoare_valid_alt in HC
     . apply hoare_valid_alt.
  intros n. induction n as [|n IH]; intros
      s HP.
  - simpl. trivial.
  - rewrite wp_at_S. split; [discriminate
     |].
    intros c' s' Hstep. inversion Hstep;
        subst.
    + apply wp_seq. eapply wp_at_post_mono
        .
      2: { apply HC; split; [exact HP |
          assumption]. }
```

```
      intros s'' HP'. apply IH. exact HP'.
           (* recurse via IH *)
    + .. (* StepWhileFalse: CSkip with
        postcondition *)
Qed.
```

The same rule, via the internal Löb. The same theorem also follows from the internal Löb rule of Section 2 without any explicit induction on n. We factor the goal $\top \vdash$ hoare P (CWhile $b\,c$) Q through the iProp $\triangleright\varphi \to \varphi$ for $\varphi :=$ hoare P (CWhile $b\,c$) Q. We prove the intermediate entailment directly, by a finite, non-recursive argument that case-analyses on the depth supplied to the `iImpl`, and close with `apply iLob`:

```
Lemma hoare_while_iLob P b c :
  hoare_valid (fun s => P s /\ beval s b =
       true) c P →
  hoare_valid P (CWhile b c) (fun s => P s
       /\ beval s b = false).
Proof.
  intros HC. rewrite hoare_valid_alt in HC
     . unfold hoare_valid.
  apply ientails_trans with
    (iLater (hoare P (CWhile b c) (...)) →
     hoare P (CWhile b c) (...)).
  - intros n _ m Hmn Hlater s HP. destruct
       m as [|m'].
    + simpl. trivial.
    + simpl in Hlater. rewrite wp_at_S.
        split; [discriminate|].
      intros c' s' Hstep. inversion Hstep;
          subst.
      * apply wp_seq. eapply
          wp_at_post_mono.
          2: { apply HC; split; [exact HP |
               assumption]. }
          intros s'' HP''. apply Hlater.
               exact HP''.  (* recurse via
               Hlater *)
      * .. (* StepWhileFalse *)
  - apply iLob.
Qed.
```

The first bullet contains no induction on n. All recursive content is hidden inside `iLob`'s proof, written once in Section 2 and not reproduced at every Hoare-style soundness proof. The six rules (skip, assign, seq, if, while, consequence) are packaged as a single theorem `imp_hoare_rules`. In the forcing-style semantics, soundness of Hoare logic over IMP is structurally identical to the soundness statement using the standard direct semantics. This is part of the point: at the first-order level the forcing reading is a different *presentation* of the same theorems. Section 4 moves to a setting where the re-presentation produces a noticeably different proof obligation.

4 Hoare Logic for an Untyped λ-Calculus with Recursion

For IMP, step-indexing was overkill: the index was a presentation convenience. We now move to a language where it is no longer optional: an untyped λ-calculus with a fix-point operator. Two things change. First, the wp predicate must explicitly encode *safety*, since the language has stuck terms (e.g. EApp (EInt 5) (EInt 3)). Second, the recursion introduced by `fix` turns the soundness predicate for a recursive function into something defined in terms of itself, so that the natural soundness proof no longer goes through meta-level induction on the step index: it goes through the internal Löb rule, used directly, with no outer induction on n.

The language. A de-Bruijn representation with integer constants, an `if0` conditional, λ-abstraction and application, and an explicit fix-point binder:

```
Inductive expr : Type :=
  | EVar  : nat → expr   | EInt  : Z →
      expr
  | EPlus : expr → expr → expr
  | EIf0  : expr → expr → expr → expr
  | ELam  : expr → expr | EApp  : expr →
      expr → expr
  | EFix  : expr → expr.
```

EFix *body* binds the self-reference at de-Bruijn index 0. We treat it as a value so it can be passed around, with self-unfolding firing only on application. The small-step relation includes the usual arithmetic and conditional rules, β-reduction EApp (ELam *body*) $v \longrightarrow body[v/0]$, and the fix-unfolding rule

```
| StepFix : forall body v, value v →
    EApp (EFix body) v ~> EApp (subst (
        EFix body) body) v.
```

which substitutes the self-reference before letting an ordinary β-reduction proceed. Reductions are call-by-value.

Step-indexed wp with safety. The wp predicate adds an explicit safety conjunct:

```
Fixpoint wp_at (n : nat) (e : expr) (Q :
    expr → Prop) : Prop :=
  match n with
  | O => True
  | S k => (value e → Q e) /\
           (value e \/ exists e', step e e
              ') /\
           (forall e', step e e' → wp_at
              k e' Q)
  end.
```

The middle conjunct rules out stuck non-value expressions, which would otherwise satisfy the wp vacuously (the first conjunct is discharged by non-valuehood, and the third quantifies over an empty step relation). Two general rules carry every soundness proof: `wp_value`, saying a value satisfies the wp targeting any Q it semantically satisfies, and the pure-step rule `wp_pure_step_at`, saying that if every reduct of e is the same e' and e' satisfies the wp at index n, then e satisfies it at $n + 1$. The pure-step rule propagates the wp through any β-reduction or fix-unfolding. We do not develop dedicated rules for `EPlus`, `EIf0`, or `EApp`. They either need an evaluation-context framework or unfold case-by-case via `wp_pure_step`, and neither is necessary for the section's conceptual point.

The specification predicate for recursive functions. The simplest non-trivial spec shape is the *closed* one, where a single predicate S plays both pre- and postcondition:

```
Definition recursive_spec (e : expr) (S :
    expr → Prop) : iProp :=
  MkProp (fun n => forall v, value v → S
      v → wp_at n (EApp e v) S)
          (recursive_spec_mono e S).
```

`recursive_spec` $e\ S$ at depth n says: "for every value v satisfying S, applying e to v is safe through n steps and produces a value satisfying S." Crucially, S appears twice. This is the source of the self-reference. To prove the spec for `EFix` *body* at depth $n + 1$ we need to know it at depth n, in order to handle the recursive call inside the body.

The* `wp_fix` *soundness rule.

Theorem 2 (`wp_fix`). *For any* body *and predicate* S*, suppose that for every value* e_{self} *satisfying* `recursive_spec` $e_{\text{self}}\ S$*, we have* `recursive_spec` (`subst` e_{self} *body*) S*. Then* $\top \vdash$ `recursive_spec` (`EFix` *body*) S.

That is, if the body is a well-behaved transformer of recursive specs, producing a function satisfying the spec from a self-reference satisfying the spec, then the fix-point itself satisfies the spec. The natural proof factors the goal through Löb's form via `ientails_trans` and closes with `apply iLob`. The finite intermediate $\top \vdash \triangleright\varphi \longrightarrow \varphi$ case-analyses on the depth supplied to the `iImpl`, uses `wp_pure_step_at` for the fix-unfolding step, and discharges the recursive obligation from the `Hlater` supplied at the predecessor index:

```
Theorem wp_fix body S :
  (forall e_self, value e_self →
     recursive_spec e_self S
       ⊢ recursive_spec (subst e_self body
           ) S) →
  ⊤ ⊢ recursive_spec (EFix body) S.
Proof.
  intros Hbody.
  apply ientails_trans with
      (iLater (recursive_spec (EFix body) S)
        → recursive_spec (EFix body) S).
  - intros n _ m Hmn Hlater. destruct m as
      [|m'].
    + intros v Hvalue HS. simpl. trivial.
    + simpl in Hlater. intros v Hvalue HS.
      (* fix-unfolding via wp_pure_step_at
         ; the recursive *)
      (* obligation is discharged from
         Hlater at m'.       *)
      apply (wp_pure_step_at m' (EApp (
         EFix body) v)
              (EApp (subst (EFix body)
                 body) v) S); [.. |
        apply (Hbody (EFix body) (VFix
           body) m' Hlater v Hvalue HS)].
  - apply iLob.
Qed.
```

No induction on n appears in this proof. All recursive content is hidden inside `iLob`, written once in Section 2. The soundness predicate `recursive_spec` is recursive in the function being verified, so the natural proof must invoke some recursive reasoning principle. A meta-level induction would have to manufacture that recursion from outside, while the internal Löb rule has it built in. Section 5 makes this structural observation precise.

Distinct pre- and postconditions. The single-predicate `recursive_spec` keeps the diagnostic of Section 5 clean. The mechanization also includes the generalised form for actual program verification:

```
Definition recursive_spec_pp (e : expr) (P
    Q : expr → Prop) : iProp :=
  MkProp (fun n => forall v, value v → P
      v → wp_at n (EApp e v) Q)
          (recursive_spec_pp_mono e P Q).

Theorem wp_fix_pp body P Q :
  (forall e_self, value e_self →
     recursive_spec_pp e_self P Q
       ⊢ recursive_spec_pp (subst e_self
           body) P Q) →
  ⊤ ⊢ recursive_spec_pp (EFix body) P Q.
```

The proof of `wp_fix_pp` is structurally identical to `wp_fix`: the same factoring through $\triangleright\varphi \longrightarrow \varphi$, the same pure-step for fix-unfolding, the same `apply iLob`. The locus argument of Section 5 carries over without change.

Worked examples. Two short verifications exercise the framework. The first is a fix-based identity, observably equal to $\lambda x.x$ but forcing a fix-unfolding at every application:

```
Definition recursive_id : expr := EFix (
    ELam (EVar 0)).
```

```
Theorem recursive_id_spec (S : expr →
    Prop) :
  ⊤ ⊢ recursive_spec recursive_id S.
```

The body ELam (EVar 0) does not mention the self-reference, so substitution is the identity and the Lipschitz premise becomes a non-recursive statement discharged by wp_pure_step_at. The second example is the canonical divergent fix-point:

```
Definition bottom : expr := EFix (EVar 0).

Theorem bottom_spec (S : expr → Prop) :
  ⊤ ⊢ recursive_spec bottom S.
Proof. apply wp_fix. intros e_self Hself.
    apply ientails_refl. Qed.
```

For bottom the body is the bare self-reference, so substitution gives e_{self} unchanged and the Lipschitz premise collapses to reflexivity. The conclusion, that bottom satisfies every recursive spec, is the familiar observation that a divergent program satisfies any partial-correctness specification vacuously. Here it appears as a one-liner because the forcing framework absorbs the divergence into the wp's index hierarchy.

5 Why the Two Proofs Look Different

We have now mechanized two Hoare-style soundness theorems against the same propositional forcing semantics: the while rule of Section 3 and the wp_fix rule of Section 4. Both are statements about recursively defined programs. Both go by some form of recursion in the proof. But the proofs look structurally different, and that difference is the main technical observation we want to articulate.

The IMP proof of hoare_while proceeds by an explicit outer induction on the step index n. The base case discharges the wp at depth 0 (which is True). The inductive step unfolds the wp at depth $n+1$, case-analyses on the step taken by CWhile b c, and in the recursive branch uses the induction hypothesis at depth n to discharge the obligation for CWhile b c itself. The proof can also be done by an appeal to the internal Löb rule, as the hoare_while_iLob alternative shows, but the two proofs are structurally identical and the choice between them is cosmetic. A precise statement of why is the three-way equivalence proved in Phase5c_Equiv.v: for any downward-closed $\Phi : \mathbb{N} \to \texttt{Prop}$ and its iProp embedding $\widehat{\Phi}$, the three propositions $\forall n.\, \Phi\, n$, $\top \vdash \widehat{\Phi}$, and $\top \vdash \triangleright\widehat{\Phi} \to \widehat{\Phi}$ are equivalent. The IMP wp_at-predicate $\lambda n.\, \forall s, P\, s \to \texttt{wp\_at}\; n\, c\, Q\, s$ is exactly of this shape, so meta-level induction-on-n and the internal-Löb route literally close the same goal.

The λ-calculus proof of wp_fix also goes by some form of recursion, since the goal is the soundness of a recursive function, yet it contains no induction on n at all. It factors the goal through Löb's form $\triangleright\varphi \to \varphi$ and lets the recursion happen inside iLob. The interior of the factored proof is finite, non-recursive step-indexed reasoning: a single case analysis on the step index supplied by the iImpl, one unfolding of the fix-application step via wp_pure_step_at, and one appeal to the Lipschitz hypothesis on the body.

Why does the difference arise? The structural reason is what we call a difference in the *recursion locus* of the soundness predicate.

For while, the wp predicate $\texttt{wp\_at}\; n\, (\texttt{CWhile}\; b\, c)\, Q\, s$ unfolds at depth $n+1$ into an obligation that mentions $\texttt{wp\_at}\; n\, (\texttt{CWhile}\; b\, c)\, Q\, s'$ at depth n for some state s'. The proposition recursive at the lower depth is *the very same proposition*, decremented in the index. At the meta level, this is just induction on n: each level of the recursion consumes one step index and we run out of indices in finitely many steps. No specifically modal reasoning is needed because the recursive obligation is parameterised solely by n.

For fix, the picture is different. The proposition $\texttt{recursive\_spec}\, (\texttt{EFix}\; body)\, S$ at depth $n+1$ unfolds, after one fix-unfolding step, into an obligation about the same $\texttt{EFix}\; body$ at depth n. It does so only because the body's behaviour at the recursive call site is guaranteed by the same proposition at depth n. So the recursive proposition appears both inside and outside the recursion: as the spec we are trying to prove for $\texttt{EFix}\; body$, and as the hypothesis we need at the recursive call. An outer induction on n can in principle reach the recursive call, since one can always recurse on n. But the proof obligation there is no longer a statement purely about smaller step indices. It is a statement about $\texttt{EFix}\; body$ itself at a smaller step index. This is exactly the shape that Löb's rule was designed to handle: "assume the spec holds later, prove it holds now", where "later" means "at a strictly smaller condition". Internalizing the recursion into Löb's rule is what frees the proof from carrying an outer induction on n.

A diagnostic. We can state this as a working slogan:

> **If the soundness predicate is itself recursive in the program**, that is, if its definition at one depth refers to its definition for the same program at a lower depth, then internal Löb is the *natural* soundness tool: it has the recursive obligation built into its conclusion, and discharging it requires no machinery beyond the rule itself. Meta-level induction on the step index remains *available*, meaning it does not become impossible, but it has to manufacture the recursive reasoning from outside the predicate it is proving, instead of finding it already inside. If the soundness predicate is only recursive in the depth, an outer induction is interchangeable with Löb and either is cheap.

The claim is a structural-naturalness one, not an impossibility one. The IMP `wp_at`, `hoare`, and `while` fall on the easy side of this diagnostic. The λ-calculus `recursive_spec` for `EFix` body with predicate S falls on the hard side, and Löb earns its keep there.

Locating the locus syntactically. The diagnostic above is informal. We have not formalized it as a metatheorem about `iProp` definitions, and we leave that as future work. But a useful working characterization, which fits the two cases we mechanize, is the following. Given a step-indexed soundness predicate $\Phi : \mathrm{nat} \to T \to \mathrm{Prop}$ (where T is the type of the program-shaped parameter: a command in the IMP setting, an expression in the λ-calculus setting), its *recursion locus is in the depth* when the recursive call in Φ's definition mentions only Φ at strictly smaller indices and at parameters that are derived from the original by small-step reduction. In this case meta-level induction on the depth has the recursive obligation already in hand, with no additional structure required. The locus is *in the program* when Φ's definition refers to Φ applied to the *same* parameter at a lower index (as `recursive_spec` (`EFix` *body*) S does, because the body's recursive call invokes the same fix-point), so that the recursive obligation cannot be discharged purely by descent on the depth: the iProp itself has to be available at the lower index as data, which is exactly what the antecedent $\triangleright\Phi$ of Löb's rule supplies. The two cases of this paper fall cleanly on the two sides of this distinction. Whether the characterization can be sharpened to a syntactic check on arbitrary iProp definitions is open.

What this says about step-indexed Hoare logic. In the verification literature, Löb's rule is sometimes presented as a convenience: a tactic available in the Iris proof mode, used when recursion is involved, but conceptually equivalent to "the step-indexed argument". Our diagnostic says this is right for first-order recursion (the IMP case) but understates what is happening in higher-order recursion (the λ-fix case). When the soundness predicate is genuinely self-referential in the program, an outer induction on n is still possible but is no longer playing to the structure of the predicate. The internal Löb rule is what plays to that structure. It is, at bottom, well-founded induction on the forcing order, internalized into the logic so that it can act on the recursive proposition directly. Section 6 will make precise what "internalized" means by showing the abstract framework's Löb rule is literally an appeal to `well_founded_ind`.

6 An Abstract Forcing Framework

The proofs of Sections 2–4 used $(\omega, \leq)$ throughout, but none required ω specifically. We abstract the framework over an arbitrary well-founded forcing notion and prove Löb's rule once at this level of generality. Two claims become precise. First, the choice of ω is a presentation convenience, not a foundational commitment. Second, the relationship between Löb's rule and well-founded induction becomes a literal identity: the abstract `iLob` is proved by `apply (well_founded_ind fs_lt_wf)`.

```
Record ForcingStructure : Type := MkFS {
  fs_cond : Type;
  fs_le : fs_cond → fs_cond → Prop;
  fs_lt : fs_cond → fs_cond → Prop;
  fs_le_refl : forall p, fs_le p p;
  fs_le_trans : forall p q r,
                  fs_le p q → fs_le q r
                    → fs_le p r;
  fs_lt_le : forall p q, fs_lt p q →
      fs_le p q;
  fs_lt_le_lt : forall p q r,
                  fs_lt p q → fs_le q r
                    → fs_lt p r;
  fs_lt_wf : well_founded fs_lt }.
```

A forcing structure is a preorder (`fs_cond`, `fs_le`) with a strict relation `fs_lt` contained in `fs_le` and right-compatible with it, such that `fs_lt` is well-founded. The compatibility axiom `fs_lt_le_lt` is what makes the later modality well-defined.

iProp and the later modality, abstractly. Over an arbitrary `FS`, `iProp` is a downward-closed predicate on `fs_cond FS`. The Heyting connectives are defined exactly as in Section 2 with $\leq$ replaced by `fs_le FS`. The interesting case is the later modality. At $(\omega, \leq)$ we defined $\triangleright\varphi$ at depth n by a match on n. At an arbitrary forcing notion we cannot match on conditions, so we give $\triangleright$ a uniform definition: it holds at p iff φ holds at every strict predecessor of p:

```
Definition iLater (φ : iProp) : iProp :=
  MkProp (fun p => forall p', fs_lt FS p'
      p → p' ⊩ φ)
        (iLater_mono_helper φ).
```

At a minimal condition this is vacuously `True`. At $(\omega, \leq)$ the new definition agrees with the old one by downward closure: "φ holds at every $m < n+1$" is, for downward-closed φ, the same as "φ holds at n."

```
Lemma iLob (φ : iProp) : (▹ φ → φ) ⊢ φ.
Proof.
  intros p. revert p.
  apply (well_founded_ind (fs_lt_wf FS)
                          (fun p => p ⊩ (▹
                              φ → φ) → p
                            ⊩ φ)).
  intros p IH Hp.
  apply Hp; [apply (fs_le_refl FS)|].
  intros p' Hp'.
```

```
    apply IH; [exact Hp'|].
    eapply imono; [apply (fs_lt_le FS);
        exact Hp' | exact Hp].
Qed.
```

Well-founded induction on p: at each p, the induction hypothesis gives φ at every strict predecessor p'. Apply `Hp` at p itself (using `fs_le_refl` to witness $p \leq p$), leaving $\triangleright\varphi$ at p to prove, which by the definition of $\triangleright$ is the quantified statement that φ holds at every strict predecessor, available from the IH. This is the literal statement of the equivalence: *Löb's rule is well-founded induction on the forcing order, internalized into the logic.*

Instances. The natural numbers with their usual order:

```
Definition nat_FS : ForcingStructure :=
  {| fs_cond := nat;
     fs_le := Nat.le;  fs_lt := Nat.lt;
     fs_le_refl := Nat.le_refl;
         fs_le_trans := Nat.le_trans;
     fs_lt_le := Nat.lt_le_incl;
     fs_lt_le_lt := Nat.lt_le_trans;
         fs_lt_wf := lt_wf |}.
```

At `nat_FS`, the abstract framework recovers Section 2: `Check iLob nat_FS` confirms that the abstract `iLob` specialized to the natural-number instance is the entailment we expect.

A non-trivially-different second instance is the pointwise product $\omega \times \omega$. Define `le_pair` $p\,q$:= `fst` $p \leq$ `fst` $q \wedge$ `snd` $p \leq$ `snd` q and `lt_pair` $p\,q$:= `le_pair` $p\,q \wedge p \neq q$. This poset is not totally ordered: $(1, 0)$ and $(0, 1)$ are incomparable. Well-foundedness of `lt_pair` follows from the measure `fst` p + `snd` p into $\mathbb{N}$ via the standard library lemma `well_founded_lt_compat`, giving `prod_FS` : `ForcingStructure`. At this instance the framework provides a step-indexed logic with two independent step counters: a natural model for two parallel components, where Löb's rule lets one reason about recursion that descends in either or both components.

A further family we do not develop but which deserves mention is the *transfinite* one. Countable ordinals up to a bound form well-founded preorders, and so does the index structure used in Transfinite Iris [Spies et al. 2021]. At any such instance, the framework's `iLob` is proved by exactly the same `well_founded_ind` on `fs_lt`. The transfinite choice produces a stronger step-indexed logic without changing the soundness machinery. The framework subsumes the index structure of Transfinite Iris in the same sense it subsumes the $(\omega, \leq)$ structure of ordinary Iris, as a particular instantiation of a generic well-founded-forcing framework.

Soundness of propositional GL.. As a concrete check, we define syntactic GL formulas `gl_form`, interpret them into `iProp` with $\Box \mapsto \triangleright$, and verify the modal core: axioms K, 4, Löb, and the necessitation rule (file `Phase5b_GL.v`). Each axiom unfolds to a small step-index calculation, with Löb's case a direct invocation of `iLob`. This confirms concretely what the topos-of-trees literature establishes abstractly: our `iProp`-with-$\triangleright$ structure is a sound model of the modal core of GL.

We have shown that the data of a step-indexed logic is captured by a `ForcingStructure` record, that the propositional fragment is parametric in this data, that the internal Löb rule is, in any such structure, a direct instance of well-founded induction, and that the standard $(\omega, \leq)$ choice is one instance, with at least one non-trivially-different alternative. The IMP and λ-calculus developments of Sections 3–4 can be lifted to be parametric in the forcing structure with mostly bookkeeping changes, not done here for readability.

7 Well-foundedness is Essential

The abstract framework of Section 6 carries one distinctive axiom: `fs_lt_wf`, the well-foundedness of the strict refinement relation. Every other axiom of `ForcingStructure` is an order-theoretic structural one: reflexivity, transitivity, compatibility. Well-foundedness is the exception, and the only axiom that uses any form of induction principle to state. It is natural to ask whether it is essential: could we drop it and still get a sensible Löb's rule for some other reason?

The answer is no, and we now make this precise. We will exhibit a "would-be" iProp framework over the integers $\mathbb{Z}$, in which the carrier and the order satisfy every part of `ForcingStructure` *except* well-foundedness. All the constructions of Sections 2 and 6 go through: we can build `iPropZ`, `iImplZ`, `iLaterZ`. But Löb's rule fails.

The setup. We work with the integers under $\leq$ and $<$ in the obvious way. Define `iPropZ` as a record of downward-closed predicates on $\mathbb{Z}$:

```
Record iPropZ : Type := MkPropZ {
  iholdZ : Z → Prop;
  imonoZ : forall p q, (q ≤ p)%Z →
      iholdZ p → iholdZ q
}.

Definition ientailsZ (φ ψ : iPropZ) : Prop
    :=
  forall p, iholdZ φ p → iholdZ ψ p.
```

The connectives follow the same definitions as before. We give the ones we need explicitly:

```
Definition iFalseZ : iPropZ :=
  MkPropZ (fun _ => False) (fun _ _ _ H =>
      H).

Definition iImplZ (φ ψ : iPropZ) : iPropZ
    :=
```

```
  MkPropZ
    (fun p => forall q, (q ≤ p)%Z →
                                iholdZ φ q →
                                    iholdZ ψ q)
    (iImplZ_mono φ ψ).

Definition iLaterZ (φ : iPropZ) : iPropZ
    :=
  MkPropZ
    (fun p => forall p', (p' < p)%Z →
        iholdZ φ p')
    (iLaterZ_mono φ).
```

The definitions are the same as in Section 6, with fs_cond := $\mathbb{Z}$, fs_le $:=\leq_{\mathbb{Z}}$, and fs_lt $:=<_{\mathbb{Z}}$. The order axioms hold: $\leq$ is a preorder, $<$ is contained in $\leq$, and $<$ is compatible with $\leq$ on the right. Only well-foundedness fails: there is no minimum and so no base case for an inductive argument.

The counterexample. Consider $\triangleright\bot$, the iProp asserting that False holds at every strict predecessor. We claim this is forced at *no* condition. Indeed, $p \Vdash \triangleright\bot$ would say "for every $p' < p$, $\bot$ holds at p'", i.e., "for every integer strictly less than p, False is true". There is at least one such integer, namely $p - 1$, and False fails there. So $\triangleright\bot$ is the empty iProp:

```
Lemma lob_premise_at p :
  p ⊩ (iImplZ (iLaterZ iFalseZ) iFalseZ).
Proof.
  intros q Hqp Hlater.
  apply (Hlater (q - 1)); lia.
Qed.
```

The lemma actually shows something more: at every p, the implication $\triangleright\bot \rightarrow \bot$ is forced. The reason is the same as just above. The antecedent $\triangleright\bot$ is empty, so any implication from it is vacuously forced. Spelled out: at p, $p \Vdash (\triangleright\bot \rightarrow \bot)$ unfolds to "for every $q \leq p$, if $q \Vdash \triangleright\bot$ then $q \Vdash \bot$". For every q, the antecedent $q \Vdash \triangleright\bot$ is itself false (apply Hlater at $q-1$, contradiction), so the conditional is vacuously satisfied.

The conclusion of Löb's rule says $\bot$ is forced at every condition. This obviously fails:

```
Lemma lob_conclusion_fails :
  ~ (forall p, iholdZ iFalseZ p).
Proof. intros H. exact (H 0). Qed.
```

Combining the two: the entailment $(\triangleright\bot \rightarrow \bot) \vDash \bot$ is the statement "for every condition, if $(\triangleright\bot \rightarrow \bot)$ is forced there then $\bot$ is forced there". The premise is forced everywhere, the conclusion is forced nowhere, so the entailment fails.

```
Theorem lob_fails_over_Z :
  ~ (iImplZ (iLaterZ iFalseZ) iFalseZ ⊨⊦
      iFalseZ).
Proof.
  intros Hent. apply lob_conclusion_fails.
  intros p. apply Hent. apply
      lob_premise_at.
Qed.
```

This concludes the counterexample: there is a forcing-like structure over $\mathbb{Z}$ in which every other piece of the abstract framework of Section 6 goes through, but Löb's rule is not derivable.

From example to characterization. The counterexample shows that well-foundedness is necessary on at least one structure. We now upgrade "at least one" to "every", mechanizing a precise iff. Fix a "pre-structure": a preorder (cond, $\leq$) with a strict relation lt satisfying all the order-theoretic axioms of ForcingStructure except well-foundedness. Build iPropPre, iImplPre, iLaterPre exactly as in Section 6. Say that *Löb holds* for the pre-structure if, for every iPropPre φ, the entailment $(\triangleright\varphi \rightarrow \varphi) \vDash \varphi$ is derivable. Then:

```
Theorem lob_iff_wf : lob_holds <->
    well_founded lt.
```

The forward direction ($\Leftarrow$) is the abstract iLob proof of Section 6, replayed in the pre-structure. The backward direction ($\Rightarrow$) is genuinely new. Define the *forcing-accessibility* predicate $\varphi_{\text{acc}}\, p :=$ Acc lt p. This is a valid iPropPre, because Acc is downward-closed along $\leq$. Its Löb premise $(\triangleright\varphi_{\text{acc}} \longrightarrow \varphi_{\text{acc}})$ is forced at *every* condition: Acc's introduction rule says exactly "every strict predecessor accessible $\Rightarrow$ this condition accessible". The assumed universal Löb then concludes φ_{acc} holds everywhere, i.e., every condition is accessible, i.e., lt is well-founded.

The statement is the step-indexed Hoare logic form of a classical soundness–completeness pairing for the Gödel–Löb axiom over transitive Kripke frames (recalled below). What is novel is the combination: stated as a property of an abstract step-indexed forcing framework, the witness is a single iProp internal to that framework (forcing-accessibility), and the result is mechanized in Coq. To our knowledge no equivalent characterisation has been recorded for any extant step-indexed Hoare logic.

Connection to Gödel–Löb. The result is the step-indexed analog of a classical observation in modal logic. The Gödel–Löb provability logic GL is sound and complete for the class of transitive Kripke frames whose accessibility relation is converse well-founded [Boolos 1993; Smoryński 1985]. Frames where the accessibility relation is *not* converse well-founded falsify the Löb axiom. The integers under $<$ are the canonical example. Our counterexample is the obvious instantiation of that classical fact into the step-indexed Hoare logic setting: forcing conditions are the worlds, strict refinement is the converse of the accessibility relation, $\triangleright$ is the necessity modality, and Löb's rule is the Löb axiom. Well-foundedness of the strict refinement is exactly converse well-foundedness of the accessibility relation.

Methodological upshot. The "ramification" in ramified forcing is not a presentation convenience to be eliminated. Shoenfield's unramification of set-theoretic forcing showed that for forcing the explicit ordinal hierarchy can be absorbed into a single recursion on rank, because the relevant induction principle is already available at the meta level. In the step-indexed setting, the analogous move is *not* available, because Löb's rule depends on a property of the forcing notion (well-foundedness) that cannot be supplied by any amount of recursion at the meta level: it has to be a property of the conditions themselves. The well-foundedness is constitutive of the modal logic and not eliminable.

Practical upshot. The choice of step-index domain decides whether Löb's rule is available. Building a step-indexed semantics on a non-well-founded index structure, such as signed counters, bidirectional approximation indices, or anything descending without bound, yields every iProp connective and a derivable $\triangleright$-monotonicity rule, but Löb's rule fails: $(\triangleright\bot) \to \bot$ becomes vacuously valid while $\bot$ stays unforced, and every soundness proof whose predicate is recursive in the program (Section 5) silently relies on a rule that does not hold. The standard candidates all pass the check: ω, products like $\omega \times \omega$, ordinals up to a bound, and Transfinite Iris [Spies et al. 2021]. Our characterization rules out only the temptation to leave the well-foundedness implicit.

8 Related Work

8.1 Step-indexing and the topos of trees

Step-indexed logical relations were introduced by Appel and McAllester [Appel and McAllester 2001] to give a semantic account of recursive types: two terms are equivalent at index n if no context running for at most n steps can distinguish them, and the desired equivalence is the intersection over all n. The downward-closure property they identify is the monotonicity of step-indexed propositions in our framework. Ahmed [Ahmed 2004] extended the technique to higher-order store, the canonical setting where step-indexing is essential rather than convenient. Birkedal and collaborators [Birkedal et al. 2011] scaled it to capability calculi. The *later* modality $\triangleright$ itself was introduced by Appel, Melliès, Richards, and Vouillon [Appel et al. 2007] to guard the step-indexed recursion syntactically.

The categorical reformulation is due to Birkedal, Møgelberg, Schwinghammer, and Støvring [Birkedal et al. 2012]: step-indexed propositions are the propositions of the internal logic of the *topos of trees*, the presheaf topos on $(\omega, \leq)$, with $\triangleright$ as a particular endofunctor and Löb's rule reflecting well-foundedness of the index domain. Subsequent work refines the categorical setting and uses it to model richer languages [Bizjak et al. 2016]. A presheaf topos on $(P, \leq)$ is the topos of sheaves for the trivial Grothendieck topology on P, the same data as a Kripke model of intuitionistic logic with worlds in P, and hence implicitly a forcing semantics in the model-theoretic sense. The word "forcing" is not, to our knowledge, prominent in the topos-of-trees literature. Making the terminological connection explicit is one of the goals of this paper.

The current state of the art on the PL side is the *Iris* family of separation logics [Jung et al. 2018; Krebbers et al. 2017] with extensions to higher-order ghost state, transfinite step-indices [Spies et al. 2021], and a substantial Coq mechanization. The internal logic of Iris is a step-indexed modal logic with $\triangleright$ and a Löb rule used pervasively as a workhorse. Related step-indexed verification systems include VST [Appel et al. 2014]. Atkey and McBride [Atkey and McBride 2013] use $\triangleright$ to characterize productive coprogramming, and Clouston, Bizjak, Grathwohl, and Birkedal [Clouston et al. 2015, 2017] develop guarded λ-calculi with $\triangleright$ as a primitive type former. Møgelberg and others [Bizjak and Møgelberg 2018; Møgelberg 2014] extend these to dependent types.

Why not build this on Iris? A natural question is whether our results could be stated as facts inside Iris. For the diagnostic of Section 5, the answer is no in practice: Iris's soundness theorem is monolithic and its internal Löb rule is used whenever a recursive predicate appears, so the distinction between recursive Hoare rules where the soundness predicate's recursion is in the depth versus in the program is invisible inside the Iris idiom. Iris does the right thing because its rule subsumes both cases, but the fact that it *has* to use the internal rule in one case and merely *may* use it in the other is not articulated in the Iris codebase or documentation that we have been able to find. For the counterexample of Section 7, Iris's model builds in well-foundedness at the level of its uPred construction, so the question "what fails if the step-index domain is not well-founded?" cannot be asked inside the framework. The minimal framework of this paper is intended not to compete with Iris but to expose the structural features of its soundness machinery in a setting small enough that the two observations are stateable.

8.2 Forcing, ramified and unramified, and the JTS translation

Cohen introduced forcing in 1963 [Cohen 1963, 1964, 1966] to prove the independence of the continuum hypothesis from ZFC. The original presentation is *ramified forcing*: the construction of the generic extension is stratified by ordinal levels mimicking Gödel's constructible hierarchy, with a "ramified language" having a type symbol for each ordinal below some bound and a forcing relation $p \Vdash_\alpha \varphi$ defined level by level. The two features that make this presentation "ramified", predicative stratification of quantification and explicit level descent, are inherited from Russell and Whitehead's ramified type theory [Russell and Whitehead 1913] and the predicative analysis tradition [Feferman 1998; Weyl 1918]. Shoenfield's 1971 reformulation [Shoenfield 1971] eliminated the syntactic ramification: forcing conditions, P-names, and the forcing relation are defined by recursion on rank and formula complexity directly. Shoenfield's presentation is equivalent and is the modern textbook standard [Jech 2003; Kunen 2011; Scott and Solovay 1967]. What was eliminated, the explicit ramified syntactic hierarchy, is exactly what we argue is *not* eliminable in the step-indexed setting.

A parallel literature in the constructive type theory community develops syntactic *forcing translations* of type theories. Jaber, Tabareau, and Sozeau [Jaber et al. 2012] give a forcing translation of CIC: for a forcing notion P, a syntactic translation takes a CIC term $M : T$ to a forced term $M^P : T^P$. For propositions, T^P is a downward-closed predicate on P: exactly our `iProp`. The line was extended in [Jaber et al. 2016] and is related to work by Coquand and collaborators on constructive forcing in type theory [Coquand and Jaber 2010].

The JTS translation is also available as a Coq plugin [Jaber et al. 12], a translation tool that develops no Hoare logic, wp predicate, or operational-semantics soundness proof. Appendix A compares it in detail. What the JTS line provides and we do not is a full translation of CIC including dependent types. Formalizing the propositional fragment of that translation against our `iProp` is the natural future direction.

8.3 The modal logic of provability

The Gödel–Löb provability logic GL is the modal logic characterized by the axiom $\Box(\Box\varphi \to \varphi) \to \Box\varphi$. Solovay's arithmetical completeness theorem [Solovay 1976] establishes that GL is sound and complete for arithmetical interpretations of $\Box$ as "provable in PA." On the model-theoretic side, GL is sound and complete for the class of transitive Kripke frames whose accessibility relation is *converse well-founded*. See Smoryński [Smoryński 1985] and Boolos [Boolos 1993].

The relevance to our work is direct. The $\triangleright$ modality of step-indexed logic is formally a GL-style $\Box$ once the strict refinement order $\prec$ on conditions is identified with the converse of the GL accessibility relation. Löb's rule of step-indexed logic is the Löb axiom of GL. The requirement that the forcing notion be well-founded is the dual of GL's requirement that the accessibility relation be converse well-founded. Our Section 7 result is a special case of the classical observation that GL is not sound on frames where that condition fails. In modal logic this is folklore. Brought into the step-indexed Hoare logic setting and mechanized, as far as we can determine, it has not previously appeared.

8.4 What this paper contributes

Each ingredient is individually known to specialists in some adjacent community: topos-of-trees to guarded domain theory, GL model theory to modal logicians, ramified forcing to set theorists, the JTS translation to type theorists. What does not appear as a single mechanized account is a step-indexed Hoare logic that articulates the recursion-locus diagnostic of Section 5 and exhibits the well-foundedness counterexample of Section 7, together with the forcing-style re-presentation that makes both stateable. We omit a survey of classical Hoare and separation logic, referring the reader to standard sources [Brookes 2007; Cook 1978; Hoare 1969; O'Hearn et al. 2001].

9 Conclusion

We have given a forcing-style semantics for step-indexed Hoare logic, mechanized in Rocq, and used it to articulate two technical observations. The first is the recursion-locus diagnostic of Section 5: when the soundness predicate is recursive only in the step index (IMP's `while`), meta-level induction on n and the internal Löb rule give structurally identical proofs, which we mechanize side by side. When it is recursive in the program itself (the λ-calculus `fix`), only the internal Löb rule plays to the predicate's structure. The second is the characterization of Section 7: in any pre-structure satisfying the order-theoretic axioms of the abstract framework, Löb's rule holds for every iProp if and only if the strict refinement relation is well-founded, the backward direction witnessed by the internal forcing-accessibility predicate $\varphi_{\text{acc}}\, p := \texttt{Acc lt}\, p$. This iff is a step-indexed Hoare logic counterpart of a classical fact about Gödel–Löb modal logic that does not, to our knowledge, appear in the verification literature. Practically, the choice of step-index domain decides whether Löb's rule is available: ordinary, transfinite, and product indices all pass the check, but well-foundedness must be verified rather than assumed.

Limitations and future directions. We work at the level of plain Hoare triples with a partial-correctness wp predicate, without separation logic, heap state, or concurrency. Whether the forcing reading scales to an Iris-scale framework is an open question this paper does not answer. Several extensions suggest themselves. One is to mechanize the type-level half of the JTS correspondence (Appendix A) as a theorem. Another is to lift the untyped λ-calculus of Section 4 to a typed source language, testing whether the recursion-locus diagnostic tracks the higher-order/first-order split or the type structure. A third is to develop a two-counter Hoare logic over the pointwise-product structure `prod_FS`, the natural setting for a concurrent calculus. The question we are most interested in is whether the recursion-locus diagnostic can be sharpened to a syntactic check on iProp definitions, applicable automatically to Iris-style soundness proofs.

The broader point is that step-indexed reasoning, viewed as a forcing construction, links three traditions that rarely meet: set-theoretic forcing, the modal logic of provability, and step-indexed verification.

A Embedding into the JTS Forcing Translation: the Propositional Fragment

A parallel literature, alongside the topos-of-trees account of step-indexing surveyed in Section 8.1, develops *syntactic* forcing translations of type theories. Jaber, Tabareau, and Sozeau [Jaber et al. 2012] give a forcing translation of the Calculus of Constructions: for any forcing notion P, they produce a syntactic translation that takes a CIC term $M : T$ to a forced term $M^P : T^P$, where the type translation $T \mapsto T^P$ corresponds to "the P-forced version of T". At the level of propositions, the forced version of Prop is essentially a downward-closed presheaf on the forcing conditions [Jaber et al. 2016], which is the type of our `iProp`.

The framework of Section 6 is therefore, up to definitional choices, the propositional fragment of the JTS forcing translation of CIC, instantiated at an arbitrary forcing structure. This identification is conceptually clean but not formally mechanized, because we have not formalized the type-level JTS translation in Rocq. What we have mechanized is the *embedding* of the unforced theory (Prop in the ambient metatheory) into the forced one (`iProp` in our framework): the constant embedding. This is the easy half of the correspondence. It makes precise what the forcing translation does to a meta-level proposition that is to be lifted into the forced logic unchanged.

Relation to the available JTS Coq plugin. The JTS translation has been implemented as a Coq plugin [Jaber et al. 12]. It is worth being explicit about how it differs from the work of this paper, because the existence of a "forcing in Coq" plugin invites the question of overlap. The plugin is a *translation tool*: it takes a Coq term $M : T$ together with a forcing structure, and produces a translated term $M^P : T^P$ in the forced universe. Its main demonstrative example is a step-indexed model of pure λ-calculus, in which the translation lets one define a `fixp` operator and prove equations such as Y-combinator reduction. The relevant infrastructure lives in the test-suite files `fix.v` and `pure_lambda_calculus.v`. The instantiation at the natural numbers with $\leq$ (`NatForcing` in the plugin's `Init.v`) produces a structure of monotone-indexed sheaves which is, up to definitional details, exactly the type of our `iProp`.

The plugin does not contain a Hoare logic, a weakest-precondition predicate, an operational semantics for any source language, or any soundness proof against an operational semantics. In particular it does not state any rule of the shape $(\triangleright\varphi \to \varphi) \vdash \varphi$ as the soundness justification for a recursive program-logic rule. The work of this paper is therefore disjoint from the plugin's contribution: we use the same forcing structure (and the same propositional fragment of the translation) that the plugin makes available, but we apply it to a different purpose: a step-indexed Hoare logic, the diagnostic of Section 5, and the well-foundedness counterexample of Section 7. The plugin would be a natural substrate to build on if one wished to formalize the *type-level* half of the correspondence we sketch here as future work.

The constant embedding. The embedding Prop $\hookrightarrow$ `iProp` sends a meta-level proposition P to the iProp that is forced everywhere if P is true and forced nowhere if P is false. Either way it is trivially downward-closed in the forcing order, since it does not depend on the condition:

```
Definition embed (P : Prop) : iProp :=
  MkProp (fun _ => P) (embed_mono P).

Notation "⌈ P ⌉" := (embed P).
```

This is the trivial part of the forcing translation: classical propositional reasoning lifts to the forcing framework without change. We will see in a moment that the genuinely new content of the forcing translation, the $\triangleright$ modality, is *not* in the image of $\lceil\cdot\rceil$.

Commutation with the Heyting connectives. The embedding is a Heyting algebra homomorphism: it commutes with each of $\top$, $\bot$, $\wedge$, $\vee$, $\to$ in both directions of entailment. The proofs are short and structural:

```
Lemma embed_True_l : ⌈ True ⌉ ⊢ iTrue.
Lemma embed_True_r : iTrue ⊢ ⌈ True ⌉.
Lemma embed_False_l : ⌈ False ⌉ ⊢ iFalse.
Lemma embed_False_r : iFalse ⊢ ⌈ False ⌉.
Lemma embed_and_intro P Q : ⌈ P /\ Q ⌉ ⊢ ⌈
    P ⌉ ∧ ⌈ Q ⌉.
Lemma embed_and_elim P Q : ⌈ P ⌉ ∧ ⌈ Q ⌉ ⊢
    ⌈ P /\ Q ⌉.
Lemma embed_or_intro P Q : ⌈ P \/ Q ⌉ ⊢ ⌈
    P ⌉ ∨ ⌈ Q ⌉.
Lemma embed_or_elim P Q : ⌈ P ⌉ ∨ ⌈ Q ⌉ ⊢
    ⌈ P \/ Q ⌉.
Lemma embed_impl_intro P Q : ⌈ P → Q ⌉ ⊢
    (⌈ P ⌉ → ⌈ Q ⌉).
Lemma embed_impl_elim P Q : (⌈ P ⌉ → ⌈ Q ⌉
    ) ⊢ ⌈ P → Q ⌉.
```

The most interesting case is the implication direction $(\lceil P\rceil \to \lceil Q\rceil) \vdash \lceil P \to Q\rceil$, where the Kripke implication of $\to$ has to be discharged by an appeal to the reflexive case "$n \leq n$". Given an inhabitant of the Kripke implication at depth n, we apply it at n itself, with the witness of $n \leq n$ given by `Nat.le_refl`, and obtain the meta-level implication. The other cases are direct unfoldings of the connectives' Kripke definitions.

Why this matters. Each commutation lemma corresponds to a clause of the propositional fragment of the JTS forcing translation. Specifically, the translation $[\![P]\!]^P$ at the level of propositions commutes with the standard connectives in exactly the way our $\lceil\cdot\rceil$ does: a translated conjunction is the conjunction of translations, and so on [Jaber et al. 2012]. Our framework therefore captures the propositional fragment of the translation. What we do not formalize is the dependent fragment, in which the translation acts non-trivially on universally quantified types and on the universes themselves. That is the part of the correspondence which would, if mechanized, turn the present informal-correspondence claim into a definitional-equality theorem.

The later modality is new content. The constant embedding is not surjective. In particular, its image is not closed under $\triangleright$: there is a meta-level proposition P such that $\triangleright\lceil P\rceil$ is not entailment-equivalent to $\lceil P\rceil$. We exhibit the witness:

```
Lemma embed_later_distinct :
  ~ (▷ ⌈ False ⌉ ⊢ ⌈ False ⌉).
Proof.
  intros H. apply (H 0). simpl. trivial.
Qed.
```

At depth 0, $\triangleright\lceil\text{False}\rceil$ is vacuously `True`, whereas $\lceil\text{False}\rceil$ at depth 0 is the constant `False`. The entailment from one to the other therefore fails at depth 0, which is exactly the witness that $\triangleright$ produces a proposition outside the embedded image.

This makes precise what the step-indexed-specific content of our framework is. The Heyting algebra structure of `iProp`, all the way through $\top, \bot, \wedge, \vee, \to$, is captured by the embedded image of Prop. What the step-indexed framework adds is the $\triangleright$ modality, and the rules that come with it (monotonicity, $\varphi \vdash \triangleright\varphi$, distribution over $\wedge$, and Löb's rule). In the JTS picture, this is the structure that comes from the forcing notion specifically, rather than from the structural machinery of the type theory. In our picture, it is the structure that comes from the well-founded strict refinement of the forcing conditions, rather than from the constant embedding.

A summary correspondence. The picture that emerges is this. Step-indexed Hoare logic, as practiced in Iris and elsewhere, has two layers of structure. The first is a Heyting-algebra fragment, which lifts from the meta-theory by a constant embedding and therefore has no forcing-specific content. The second is a modal fragment ($\triangleright$, `iLob`, and the rules around them), which is specifically the structure that the well-founded forcing notion contributes. The JTS forcing translation is the type-theoretic analog of this picture: for any forcing notion P, it gives a translation of CIC that adds P-specific structure to the embedded image of the unforced theory. At the level of propositions, the correspondence we formalize is the constant embedding. A full correspondence at all levels of the universe would require mechanizing the JTS translation itself, which we leave as future work.

References

Amal Ahmed. 2004. *Semantics of Types for Mutable State.* Ph. D. Dissertation. Princeton University.

Andrew W. Appel et al. 2014. *Program Logics for Certified Compilers.* Cambridge University Press. The VST (Verified Software Toolchain) framework..

Andrew W. Appel and David A. McAllester. 2001. An Indexed Model of Recursive Types for Foundational Proof-Carrying Code. *ACM Trans. Program. Lang. Syst.* 23, 5 (2001), 657–683.

Andrew W. Appel, Paul-André Melliès, Christopher D. Richards, and Jerome Vouillon. 2007. A Very Modal Model of a Modern, Major, General Type System. *ACM SIGPLAN Notices (POPL)* 42, 1 (2007), 109–122. Often referred to as "the very-modal model"; introduces ▷ for step-indexed logical relations..

Robert Atkey and Conor McBride. 2013. Productive Coprogramming With Guarded Recursion. In *ICFP.* 197–208.

Lars Birkedal, Rasmus Ejlers Møgelberg, Jan Schwinghammer, and Kristian Støvring. 2012. First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees. *Logical Methods in Computer Science* 8, 4 (2012).

Lars Birkedal, Bernhard Reus, Jan Schwinghammer, Kristian Støvring, Jacob Thamsborg, and Hongseok Yang. 2011. Step-Indexed Kripke Models over Recursive Worlds. In *POPL.* 119–132.

Aleš Bizjak, Hans Bugge Grathwohl, Ranald Clouston, Rasmus Ejlers Møgelberg, and Lars Birkedal. 2016. Guarded Dependent Type Theory With Coinductive Types. In *FoSSaCS.* 20–35.

Aleš Bizjak and Rasmus Ejlers Møgelberg. 2018. Denotational Semantics for Guarded Dependent Type Theory. In *FoSSaCS.*

George S. Boolos. 1993. *The Logic of Provability.* Cambridge University Press.

Stephen Brookes. 2007. A Semantics for Concurrent Separation Logic. *Theoretical Computer Science* 375, 1–3 (2007), 227–270.

Ranald Clouston, Aleš Bizjak, Hans Bugge Grathwohl, and Lars Birkedal. 2015. Programming and Reasoning With Guarded Recursion for Coinductive Types. In *FoSSaCS.* 407–421.

Ranald Clouston, Aleš Bizjak, Hans Bugge Grathwohl, and Lars Birkedal. 2017. The Guarded Lambda-Calculus: Programming and Reasoning With Guarded Recursion for Coinductive Types. *Logical Methods in Computer Science* 12, 3 (2017).

Paul J. Cohen. 1963. The Independence of the Continuum Hypothesis. *Proc. National Academy of Sciences* 50 (1963), 1143–1148.

Paul J. Cohen. 1964. The Independence of the Continuum Hypothesis, II. *Proc. National Academy of Sciences* 51 (1964), 105–110.

Paul J. Cohen. 1966. *Set Theory and the Continuum Hypothesis.* W. A. Benjamin.

Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90.

Thierry Coquand and Guilhem Jaber. 2010. A Note on Forcing and Type Theory. *Fundamenta Informaticae* 100 (2010), 43–52.

Solomon Feferman. 1998. *In the Light of Logic.* Oxford University Press.

C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580.

Guilhem Jaber, Gabriel Lewertowski, Pierre-Marie Pédrot, Matthieu Sozeau, and Nicolas Tabareau. 2016. The Definitional Side of the Forcing. In *LICS.* 367–376.

Guilhem Jaber, Matthieu Sozeau, and Nicolas Tabareau. 2012–. Forcing: A Forcing Layer on Top of Coq. GitHub repository, https://github.com/mattam82/Forcing.

Guilhem Jaber, Nicolas Tabareau, and Matthieu Sozeau. 2012. Extending Type Theory With Forcing. In *LICS.* 395–404.

Thomas Jech. 2003. *Set Theory* (3rd ed.). Springer.

Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris From the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic. *Journal of Functional Programming* 28 (2018).

Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. 2017. The Essence of Higher-Order Concurrent

Separation Logic. In *ESOP*. 696–723.
Kenneth Kunen. 2011. *Set Theory*. College Publications.
Rasmus Ejlers Møgelberg. 2014. A Type Theory for Productive Coprogramming Via Guarded Recursion. In *CSL-LICS*.
Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. 2001. Local Reasoning About Programs That Alter Data Structures. *CSL* (2001).
Bertrand Russell and Alfred N. Whitehead. 1910–1913. *Principia Mathematica*. Cambridge University Press.
Dana Scott and Robert M. Solovay. 1967. Boolean-Valued Models for Set Theory. In *Proc. American Math. Soc. Summer Inst. on Axiomatic Set Theory*. Unpublished; Boolean-valued model presentation of forcing..
Joseph R. Shoenfield. 1971. Unramified Forcing. *Axiomatic Set Theory, Proc. Symp. Pure Math.* 13 (1971), 357–381.
Craig Smoryński. 1985. *Self-Reference and Modal Logic*. Springer.
Robert M. Solovay. 1976. Provability Interpretations of Modal Logic. *Israel J. Math.* 25 (1976), 287–304.
Simon Spies, Niklas Mück, Hoang-Hai Dang, Manuel Rigger, Michael Sammler, Joseph Tassarotti, Robbert Krebbers, and Derek Dreyer. 2021. Transfinite Iris: Resolving an Existential Dilemma of Step-Indexed Separation Logic. *PLDI* (2021). Extends Iris with transfinite step indices..
Hermann Weyl. 1918. *Das Kontinuum*. Veit, Leipzig. Translated as "The Continuum: A Critical Examination of the Foundation of Analysis," Dover, 1994..